

Lab: 01 Build a Recurrent neural network:

Aim:

To build and train a Recurrent neural network:

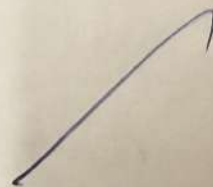
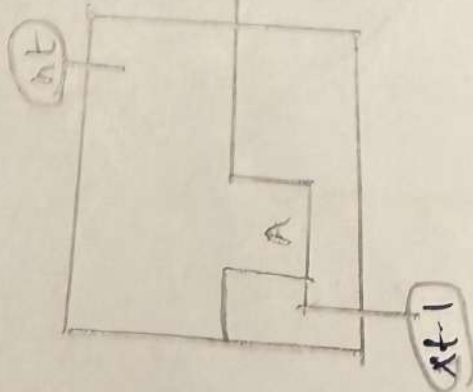
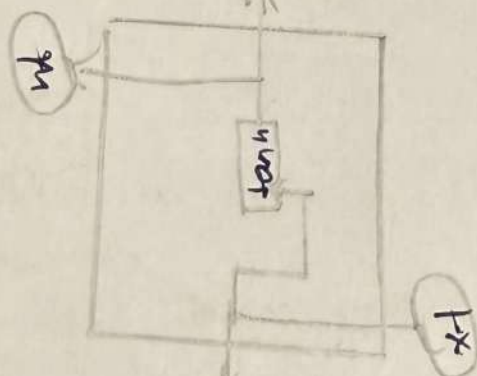
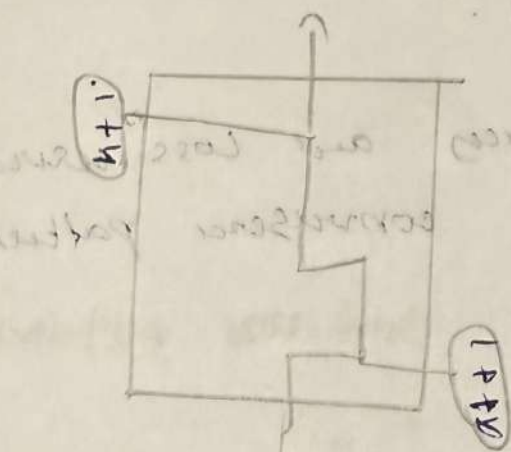
Objective:

1. To study the arch of Recurrent neural network.
2. To implement a simple RNN model using Tensorflow.
3. To compare the RNN's performance with LSTM.
4. To visual and analyze.

Pseudocode:

1. Import necessary libraries (tensorflow/layers, numpy, matplotlib).
2. Load and preprocess a sequence dataset.
3. Convert input sequence to padded vector.
4. Define RNN model.
5. compile model:
6. Train model using training data.
7. Evaluate model performance and visualize

RNN Arch.



Observation:

1. Training Analysis: The RNN learned to map input sequences to sentiment labels, but training took longer compared to feedforward networks.

2. Performance: Accuracy of RNN was moderate (~80%) slightly lower than LSTM.

~~3. Model Training.~~

3. Loss and Accuracy: Both training and loss decreased over epochs.

4. Limitation and Insight: Standard RNN suffered from vanishing gradient problem when dealing with long sequences.

Result:

80% successfully implemented a Recurrent neural network to analyze sequential data.

epoch 1/6	Train loss: 0.6432	Test loss: 0.6432
epoch 2/6	Train loss: 0.5635	Test loss: 0.3960
epoch 3/6	Train loss: 0.2993	Test loss: 0.3174
epoch 4/6	Train loss: 0.1967	Test loss: 0.3174
epoch 5/6	Train loss: 0.1360	Test loss: 0.2307
epoch 6/6	Train loss: 0.0780	Test loss: 0.3000

